

Foundations of Cybersecurity/Applied Cryptography

16 January 2024

Exercise n. 1 [12] [All]

1. Illustrate the CBC scheme.
2. Argue the advantages of the CBC scheme w.r.t. the ECB scheme.
3. Let us assume that at encryption time there is an error in the i -th plaintext block P_i . Answer the following questions for both the ECB and CBC mode:
 - a. Which and how many ciphertexts are affected?
 - b. What's the effect experimented by the receiver?

Exercise n. 2 [10] [All]

Consider the following authentication and key agreement protocol:

M1 $A \rightarrow B: \{A, B, n_A, \text{pwd}_{AB}, K'\}_{K_B}$

M2 $B \rightarrow A: \{A, B, n_A\}_{K'}$

where K_B is Bob's public key, pwd_{AB} is a password that Alice and Bob manage to share by off-line means, n_A is a nonce, and K' is the session key.

1. Analyse the protocol and specify the assumptions under which it fulfils the key authentication and key confirmation requirements.
2. Discuss whether n_A is necessary or if it can be removed from the protocol.
3. Argue whether the protocol is subject to a replay attack.
4. In case of positive answer to the previous question, modify the protocol to remove the vulnerability.

Exercise n. 3 [8] [2022-23]

Let G be a generator of the elliptic curve $\mathcal{E}$. Consider the following digital signature scheme.

- *Key Generation.* Randomly select x_A in $\mathbb{Z}_p$ and computes $Y_A = x_A \cdot G$. Let x_A and Y_A be Alice's private and public key respectively.
- *Signature generation.* Alice digitally signs message M as follows: i) she selects a random number k in $\mathbb{Z}_p$, and 2) computes the digital signature $S = M - k \cdot x_A \cdot G$. Then, Alice transmits Bob the bundle (M, k, S) .
- *Signature verification.* Upon receiving the bundle, Bob verifies the digital signature by checking whether the following equality is valid: $M = S + k \cdot Y_A$.

Question 1. Show that the scheme actually works, i.e., show that the signature verification process produces an equality if the signature is valid.

Question 2. Show that the scheme is subject to existential forgery.

Exercise n. 3 [8] [earlier]

Consider the following piece of C code that senselessly copies the first command-line argument into a local C-string:

```
int main(int argc, char* argv[]) {
    char opt1[128];
    if (argc < 2) {
        /* Handle error */
    }
    strcpy(opt1, argv[1]);
    return 0;
}
```

Does the code contain any vulnerability? If so, give an example of how an attacker can exploit it. How should the code be corrected?

Solution

EXERCISE N.1

Question 1 and 2: see theory.

Question 3. Because of the error, P_i' is encrypted instead of P_i . As to ECB, only C_i is affected whereas in CBC C_i and all the following ciphertext blocks are affected. However, in both cases the receiver correctly obtains the original plaintext blocks where, of course, P_i' is in the place of P_i .

EXERCISE N.2

Question 1. The main assumption is that B believes K' fresh.

Question 2. If A generates fresh session keys, then n_A may be removed.

Question 3. Because of the assumption at point 1), an adversary that manages to obtain a session key K' then (s)he can impersonate Alice with respect to Bob.

Question 4. A possible solution is the following:

M1 $B \rightarrow A: n_B$

M2 $A \rightarrow B: \{A, B, n_B, \text{pwd}_{AB}, K'\}_{K_B}$

M3 $B \rightarrow A: \{A, B\}_{K'}$

EXERCISE N.3 [2022-23]

Question 1. Bob computes $S + k \cdot Y_A = (M - k x_A G) + k \cdot Y_A = (M - k Y_A) + k Y_A = M$.

Question 2. Adversary selects S , selects k , and then computes $M = S + k Y_A$. Then, (s)he releases (M, k, S) .

EXERCISE N.3 [EARLIER]

All the `argv[*]` arguments are assured to contain a string terminators. However, the length of `argv[1]` could go beyond the capacity of `opt1`, causing a buffer overflow. A solution is to use the `strncpy()` standard function in place of `strcpy()`, which lets the programmer specify the maximum number of characters to copy. The `strncpy()` function has the peculiarity that, if the maximum number of copied characters is copied, the destination string is left without terminator. This could lead to further vulnerabilities. To avoid non-terminated strings, it is always needed to “manually” put a terminator at the end of the destination buffer. The corrected code is the following:

```
int main(int argc, char* argv[]) {
    char opt1[128];
    if (argc < 2) {
        /* Handle error */
    }
    strncpy(opt1, argv[1], 128);
    opt1[127] = '\0';
    return 0;
}
```